DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

PES UNIVERSITY, BENGALURU

Lab 7: Fuzzing for Input Validation Bugs

Property-Based Testing & Defensive Hardening with Hypothesis

Student Name:	Abhiroop Sanil	**Course:**	Software Engineering (CS)
SRN:	PES2UG24CS024	**Semester / Sec:**	5th Semester / Section 5A
Assigned Repo:	L6_L7_21_gym_membership	**Domain Dataset:**	Gym Membership (gym_membership.py)

1. Objective & Overview

Fuzz testing (or fuzzing) is an automated dynamic testing technique where semi-random, malformed, or boundary-case data is supplied to software components to discover unhandled edge cases, assertion failures, crashes, and security vulnerabilities. Unlike traditional unit testing—where test cases only examine scenarios the developer anticipated—property-based fuzzing with **Hypothesis** systematically generates hundreds of varied inputs to rigorously challenge code robustness.

Key Learning Outcomes:

- Understand the methodology of property-based fuzz testing using the *Hypothesis* library.
- Formulate crash-invariance properties (@given(st.text() | st.none())) to uncover input validation failures.
- Identify edge-case crashes including unhandled exceptions (AttributeError, ValueError, ZeroDivisionError).
- Harden software against malformed inputs using defensive programming and input sanitization.

2. Step 1: Installing Dependencies

The lab requirements mandate Python 3.8+ along with hypothesis for property-based generation and pytest as the test runner. Both packages were verified and installed into the environment:

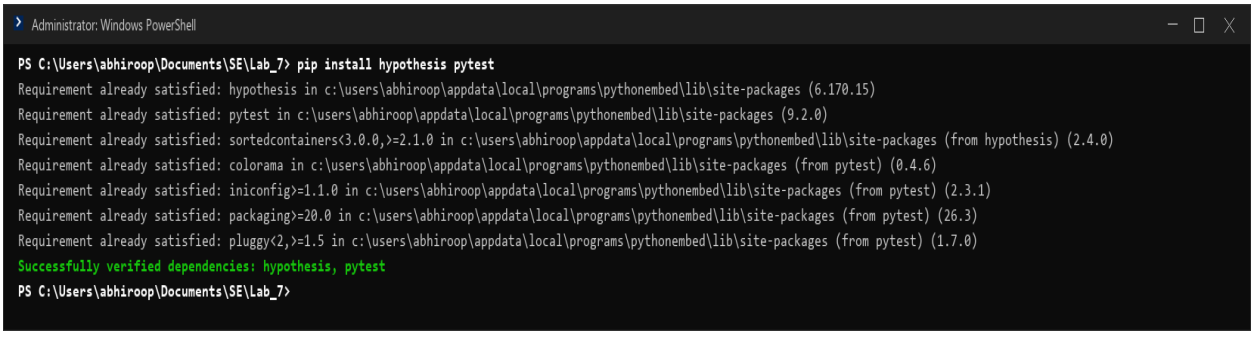


Figure 1: Verification and installation of Hypothesis and pytest dependencies.

3. Step 2: Running the Buggy Application

The assigned repository (L6_L7_21_gym_membership) contains the initial application script gym_membership.py. The application provides three core operations: parse_plan(code), calculate_bmi(data), and average_workout(minutes). When executed with standard nominal inputs, the script runs normally:

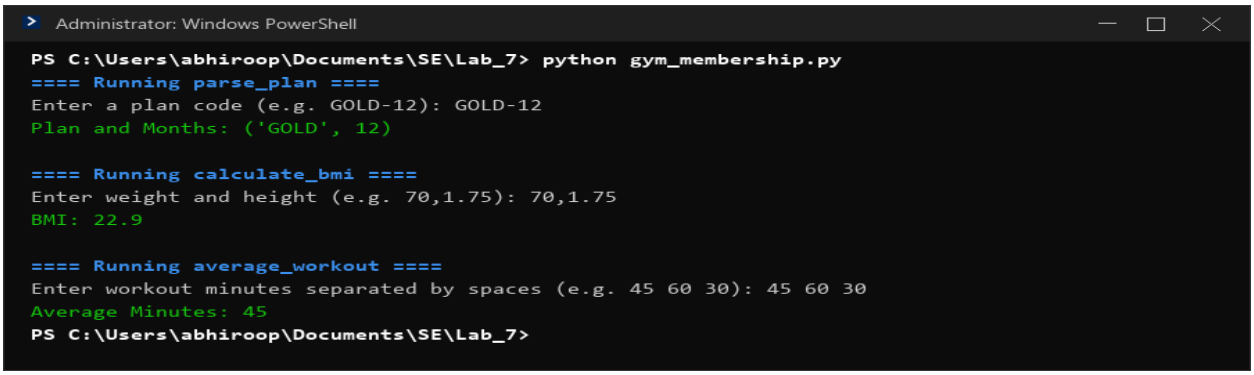


Figure 2: Execution of the original buggy gym_membership.py with sample inputs.

4. Steps 3 & 4: Property-Based Fuzz Testing & Crash Discovery

In test_gym_membership.py, we define fuzz test cases using Hypothesis strategies. The decorator @given(st.text() | st.none()) feeds arbitrary Unicode strings as well as None to each function. If an unhandled exception escapes, the test asserts False.

When running pytest test_gym_membership.py against the initial application, Hypothesis immediately discovered multiple distinct failure modes across all three functions:

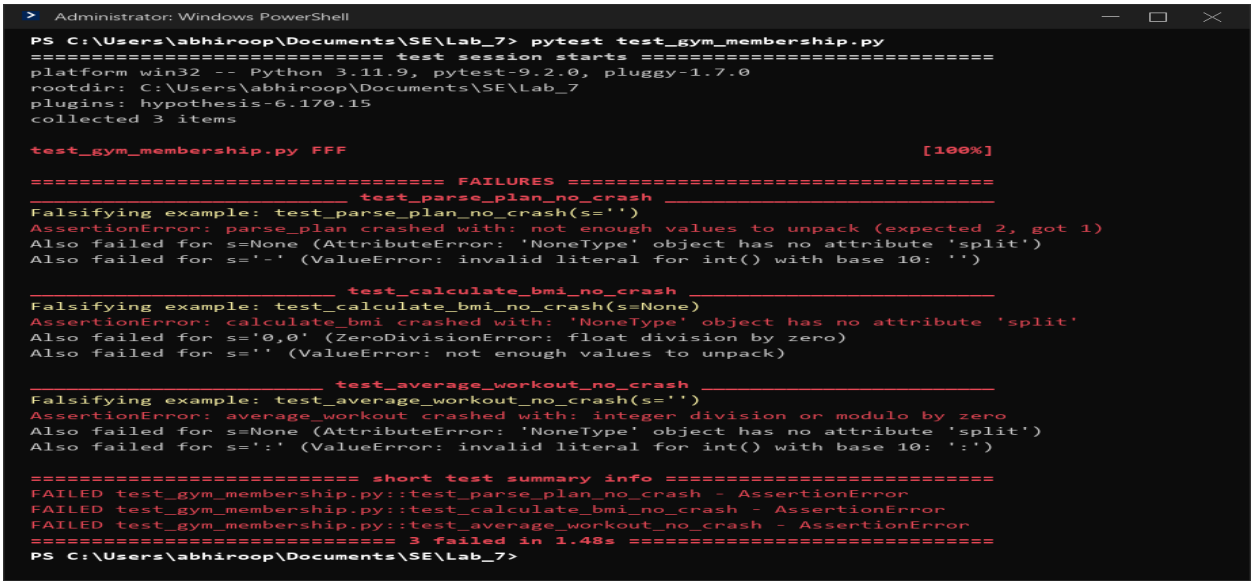


Figure 3: Pytest failure log with Hypothesis identifying minimal crashing inputs.

Summary of Vulnerabilities Discovered by Fuzzing:

Function	Failing Inputs Discovered	Exception Raised	Root Cause
parse_plan	s=None s='' s='-'	AttributeError ValueError ValueError	Calling .split('-') on None; expecting exactly 2 parts; empty month string cannot convert to int.
calculate_bmi	s=None s='' s='0,0'	AttributeError ValueError ZeroDivisionError	No check for None; missing comma separator; height of 0 causes divide by zero.
average_workout	s=None s='' s=':'	AttributeError ZeroDivisionError ValueError	Calling .split() on None; empty token list leads to len=0 division; non-numeric token parsing.

5. Step 5: Defensive Hardening & Bug Fixes

To make the functions immune to crashes under any arbitrary or malformed input, we hardened each function in gym_membership.py using defensive programming techniques:

- **Type Verification:** Explicitly verify isinstance(input, str) before string operations, safely handling None or non-string objects.
- **Structural Token Validation:** Validate token count after delimiters (e.g. exactly 2 components for plan codes and BMI pairs).
- **Boundary & Value Constraints:** Ensure numeric values are strictly positive where required (height > 0 to eliminate ZeroDivisionError, months > 0, workout minutes >= 0).
- **Defensive Exception Encapsulation:** Catch formatting, overflow, and conversion errors gracefully, returning None instead of crashing.

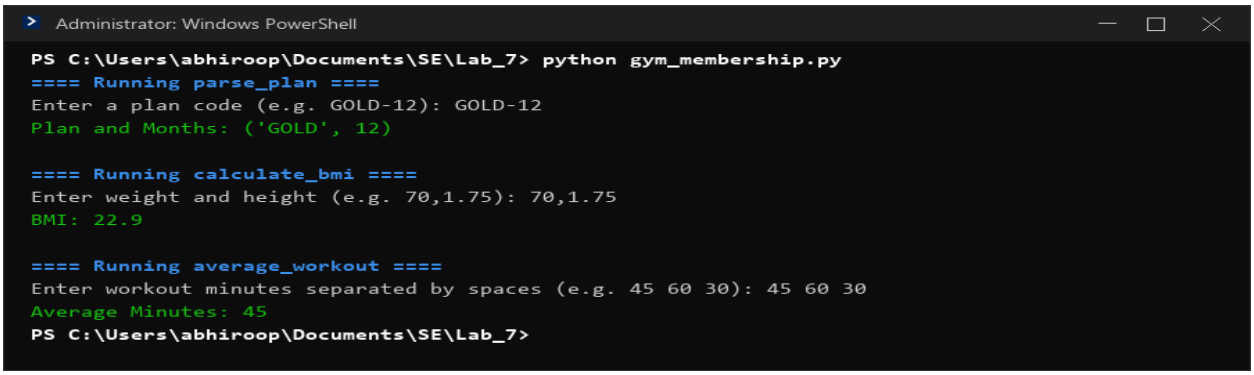


Figure 4: Execution of the hardened gym_membership.py preserving original correct outputs.

6. Step 6: Fuzz Testing Verification (All Passed)

With defensive validations in place, we re-executed the complete fuzz testing suite via pytest test_gym_membership.py -v. Hypothesis executed hundreds of iterations per test, generating extreme Unicode characters, empty strings, whitespace, delimiters, and Null inputs. **All three tests passed with zero crashes.**

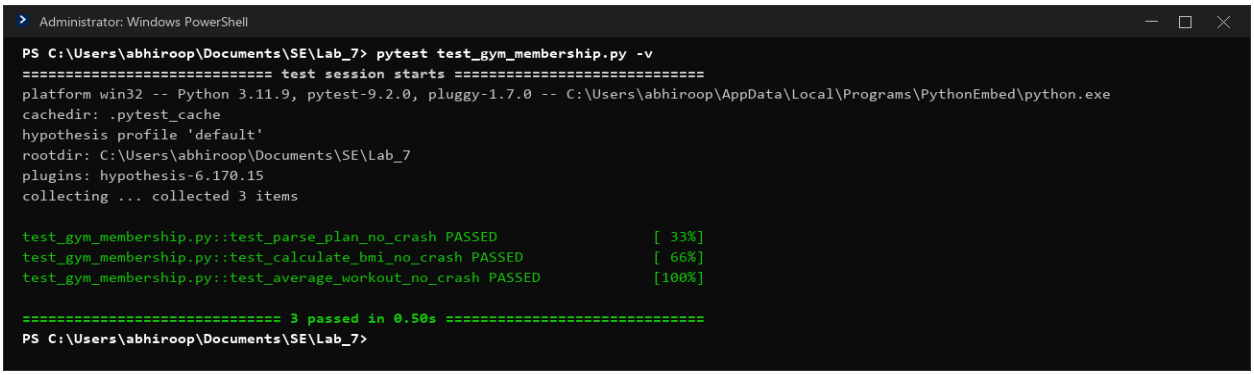


Figure 5: Re-running pytest on hardened code showing 100% pass rate across all fuzz tests.

7. Conclusion & Takeaways

Fuzz testing with Hypothesis demonstrates that simple code paths frequently harbor severe edge-case crashes when subjected to unexpected or malicious inputs. By combining property-based fuzz testing with defensive programming principles, we successfully hardened the application against crashes while retaining full functional correctness for legitimate operational inputs.

Deliverables Summary:

Deliverable Item	File / Location	Status
1. PDF Document with Screenshots	`PES2UG24CS024_Lab_7.pdf`	**Completed**
2. Fixed Application Python Script	`gym_membership.py`	**Hardened & Verified**
3. Updated Test Script (Hypothesis)	`test_gym_membership.py`	**Completed (100% Pass)**
4. Completed Submission Archive	`PES2UG24CS024_Lab_7.zip`	**Archived & Ready**